SourceKit

BETA V2.0 / EARLY ACCESS

Find engineers by what they build.

AI-powered technical sourcing, grounded in real GitHub data.

getsourcekit.vercel.app

OVERVIEW

What is SourceKit?

SourceKit is a technical sourcing tool that finds software engineers based on their actual code contributions on GitHub, not self-reported LinkedIn keywords. You enter a role and company. SourceKit builds a sourcing strategy, searches real repositories, and returns a scored pipeline of candidates in minutes. v2.0 adds Parallel.ai for intelligent JD extraction and Exa Websets for persistent, auto-updating candidate collections.

Signal Over Noise

Candidates are scored on real GitHub activity: commits, pull requests, and repo contributions. Ground-truth engineering signal that resumes can't replicate.

AI-Built Strategy

Claude analyzes the role, generates target repos, identifies poach companies, and builds a skills matrix. Parallel.ai extracts JDs from any URL automatically.

Minutes, Not Days

Job description to scored candidate pipeline in under two minutes. No boolean strings. No manual sourcing. No tool switching.

WHY SOURCEKIT

How is this different?

Traditional Sourcing

Search LinkedIn by keywords. Filter by title and company. Hope the resume matches reality. High noise, profile-optimized results, limited engineering signal.

SourceKit Sourcing

AI identifies key repos for a role, searches GitHub for top contributors, scores candidates on commit history, languages, and impact. Parallel.ai adds web context. Exa Websets build living candidate lists. Evidence-based, not keyword-based.

Best Use Cases

Niche and senior technical roles. Passive candidates not actively job-seeking. Engineers at early-stage startups. Anyone with a meaningful open-source presence.

UNDER THE HOOD

Architecture and Technology

SourceKit combines five core technologies: Claude AI for intelligence, Exa for neural search and Websets, Parallel.ai for web search and content extraction, the GitHub API for contributor data, and Supabase for data persistence.

PRESENTATION LAYER

React + TypeScript Web App
Mobile-first UI / Google OAuth / Settings / Search / Pipeline / Websets

▼

AI INTELLIGENCE LAYER

Claude AI (Anthropic)
Strategy / Scoring / Outreach

Exa Search + Websets
Neural search / Websets / Candidate collections

Parallel.ai
Web search / JD extraction / Enrichment

GitHub REST + GraphQL
Contributors / Commits / Graphs

▼

DATA LAYER

Supabase (PostgreSQL)
Auth / Row-level Security / User Data / Pipeline State / Webset Mappings

Enrichment APIs
LinkedIn matching / Email discovery / Profile enrichment

▼

INFRASTRUCTURE

Vercel
Hosting / Serverless Functions / Edge Network

Stripe
Billing / Usage Tracking / Tier Management

HOW IT WORKS

Workflow: From Role to Pipeline

You Input
Role + company, job description, or job URL

▶

AI Strategy
Claude builds target repos, poach list, skills matrix

▶

You Refine
Edit repos, add or remove targets, adjust criteria

▶

Multi-API Search
Exa + Parallel + GitHub APIs scan for top contributors

▶

Scored Pipeline
Candidates ranked by code signal, ready for outreach

GETTING STARTED

Three Ways to Start a Search

1. Role + Company

Type a job title and target company. Claude infers the technical stack, identifies relevant repos, and builds your strategy from scratch. Example: "Staff ML Engineer" at "Anthropic"

2. Paste a Job Description

Copy and paste the full JD text. The AI parses requirements, technologies, and seniority level to generate a targeted strategy.

3. Paste a Job URL

Drop in a link from Lever, Greenhouse, or Ashby. Parallel.ai extracts the JD content and builds the strategy automatically. Works with JS-rendered pages.

SCORING

How Candidates Are Scored

Every candidate receives a composite score from 0 to 100 based on actual GitHub activity, weighted by relevance to the target role. No self-reported data. Just code signal.

Strong Match 80-100	Worth Reviewing 60-79	Lower Signal 0-59

Commit Volume and Recency

How actively and recently the candidate has contributed to relevant repos. Recent activity is weighted more heavily because it indicates current working knowledge.

Language and Stack Match

How well the candidate's primary languages align with what the role requires. Python for ML roles, Rust for systems work, and so on.

Repo Relevance and Impact

Contributions to high-signal repos (well-starred, actively maintained) are weighted more heavily than activity in personal or low-visibility projects.

PIPELINE

5-Stage Candidate Pipeline

Sourced ▶ Contacted ▶ Responded ▶ Screen ▶ Offer

Bulk Actions

Select multiple candidates to compare side-by-side, generate a summary report, or batch-draft personalized outreach messages.

CSV Export

Export your results or pipeline at any time. Take your candidate data into your ATS, spreadsheets, or any other tool.

EXA WEBSETS NEW IN V2.0

Exa Websets: Persistent Candidate Collections

Websets are persistent, auto-updating candidate collections powered by Exa's neural search. Define your criteria once and get new matches continuously as new contributors emerge.

Create and Search

Build a Webset from any search or from scratch. Define entity type, search query, and filtering criteria. Exa finds matching candidates across the web.

Auto-Enrich

Every item is automatically enriched with current company, role, LinkedIn URL, GitHub profile, and primary programming languages. No manual lookup.

Monitor

Set a schedule and Websets auto-update with new matches. New candidates matching your criteria appear without re-running searches. Push matches to your pipeline.

BETA TESTING

What to Test

1. **Sign In** — Log in with Google at getsourcekit.vercel.app
2. **Build a Strategy** — Try all three input paths: role + company, pasted JD, or job URL
3. **Edit and Run the Search** — Review the AI-generated repo list. Add, remove, or rearrange, then run the search.
4. **Filter Results** — Narrow by language, minimum commits, star count, and location
5. **Review Candidates** — Check candidate cards and scores. Click into profiles. Try "Find LinkedIn" and "Enrich" on your top matches.
6. **Test Exa Websets** — Create a Webset from search results. Add criteria. Check enrichment data. Set up monitoring.
7. **Pipeline and Bulk Actions** — Add candidates to the pipeline. Drag between stages. Select multiple for compare, summarize, and outreach.
8. **Export to CSV** — Export and verify that the data looks clean and complete

PRO TIPS

Best Practices

1. Be Specific With Titles

Use "ML Infrastructure Engineer focused on training pipelines" instead of just "ML Engineer." Specificity dramatically improves repo targeting and result quality.

2. Always Edit the Repo List

The AI-generated suggestions are a starting point. Editing the repo list is the single biggest lever you have for improving result quality.

3. Target Hidden Gems

Candidates with high contributions but low follower counts tend to be under-recruited and more responsive to outreach. Prioritize them.

4. Run Multiple Searches

Search the same role across different companies to widen your net. Different target companies surface different repo ecosystems and candidate pools.

5. Use Websets for Ongoing Roles

For roles you hire repeatedly, set up a Webset with monitoring. New contributors matching your criteria surface automatically.

GOOD TO KNOW

Known Limitations

LIMITATION	IMPACT	WORKAROUND
GitHub-only sourcing	Misses engineers not active on GitHub	Use alongside LinkedIn Recruiter for full coverage
Strategy build time (15-20s)	Brief wait while AI generates the plan	Expected behavior. Full search takes 30-60s.
JS-rendered job pages	Some job URLs may still fail to parse	Paste the JD text directly instead of the URL
LinkedIn URL accuracy	Links are AI-matched, not guaranteed	Always verify the LinkedIn profile before outreach
Websets sync latency	New monitored matches may take minutes	Check back periodically or set monitoring cadence

Feedback Welcome

Bugs, bad searches, confusing UX, missing features, AI quality issues. All fair game. Screenshots help.

Mike Rubino / michael.f.rubino@gmail.com